Project 1 Solutions
EECE 5614
Student Name: Rimsha Kayastha

Part 1
Main coding part:

```python
class EpsilonGreedy(UtilityFunctions):
    """Part 1"""
    def part1(self):
        average_Q_a_values, average_Q_b_values, average_acc_rewards_epsilon = self.rl_bandit_100loops(self.part1_combi

        epsilon_greedy_results = {}
        for rewards, lr in zip(average_acc_rewards_epsilon, self.part1_combinations_str):
            epsilon_greedy_results[lr] = rewards
        self.part1_results = epsilon_greedy_results

        # Plot - 4 subplots (one per learning rate)
        fig, axes = plt.subplots(2, 2, figsize=(14, 10))
        axes = axes.flatten()

        colors = ['blue', 'orange', 'green', 'red']

        for α_idx in range(4):  # 4 learning rates
            ax = axes[α_idx]

            for ε_idx, (ε, color) in enumerate(zip(self.part1_epsilon_policies, colors)):
                # Index into average_acc_rewards
                combo_idx = α_idx * 4 + ε_idx
                avg_acc_rewards = average_acc_rewards_epsilon[combo_idx]

                ax.plot(range(1, len(avg_acc_rewards)+1), avg_acc_rewards,
                        label=f'ε={ε}', color=color, linewidth=1.5)

            ax.set_xlabel('Time(t)', fontsize=12)
            ax.set_ylabel('Average Accumulated Reward', fontsize=12)
            ax.set_title(f'{self.part1_learning_rates_str[α_idx]}', fontsize=14, fontweight='bold')
            ax.legend(loc='best')
```

Figure 1: Graphs of average accumulated rewards across time for each learning rate and epsilon values:

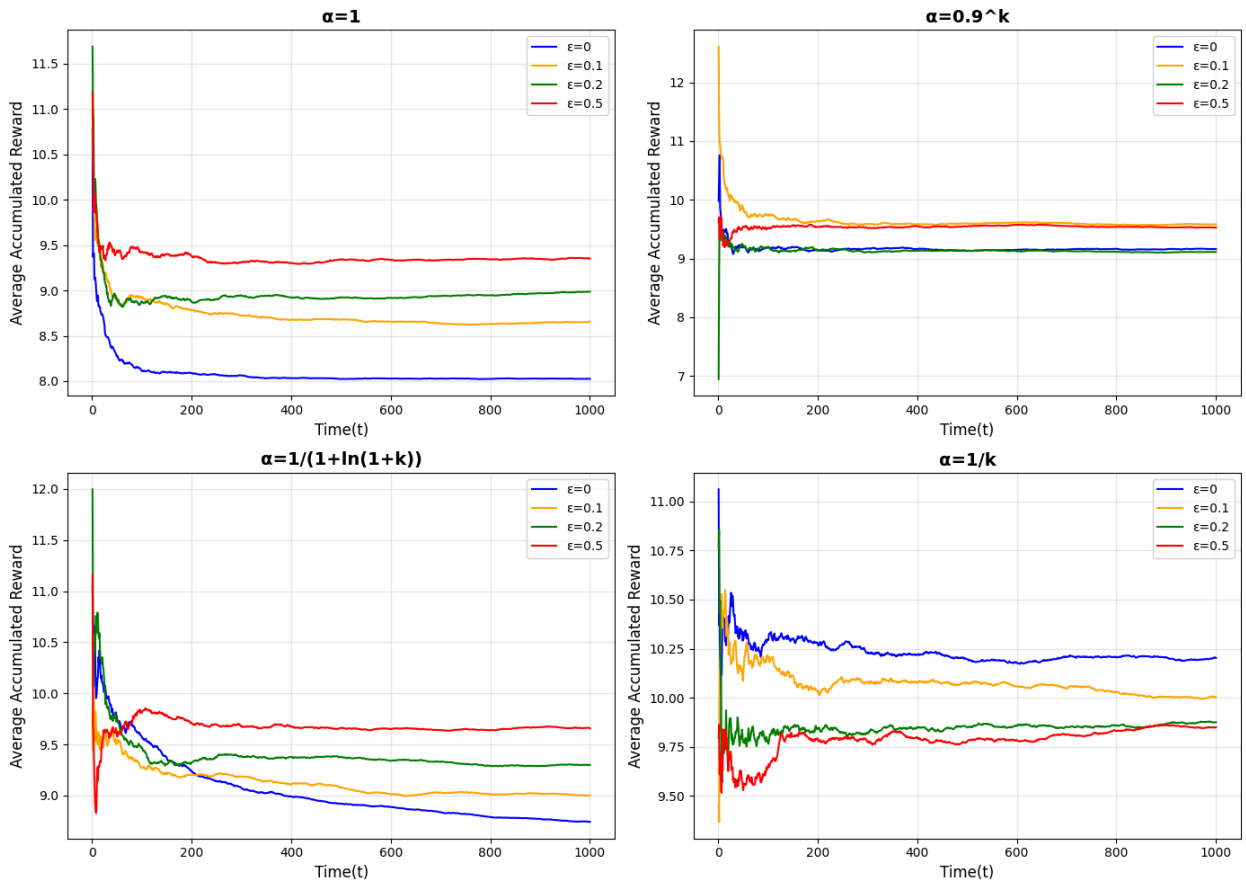


Table 1: Average Q-values for α = 1

Epsilon-greedy	Average of action value Q(a1) of 100 runs	True action value Q∗(a1)	Average of action value Q(a2) of 100 runs	True action value Q∗(a2)
ϵ = 0 (greedy)	8.01	8	-14.20	11
ϵ = 0.1	6.53	8	-0.83	11
ϵ = 0.2	6.68	8	-1.07	11
ϵ = 0.5	7.37	8	3.86	11

Table 2: Average Q-values for α = 0.9^k

Epsilon-greedy	Average of action value Q(a1) of 100 runs	True action value Q∗(a1)	Average of action value Q(a2) of	True action value Q∗(a2)

			100 runs	
ϵ = 0 (greedy)	5.00	8	0.18	11
ϵ = 0.1	5.28	8	5.04	11
ϵ = 0.2	6.79	8	4.36	11
ϵ = 0.5	7.54	8	8.32	11

Table 3: Average Q-values for α = 1/(1+ln(1+k))

Epsilon-greedy	Average of action value Q(a1) of 100 runs	True action value Q∗(a1)	Average of action value Q(a2) of 100 runs	True action value Q∗(a2)
ϵ = 0 (greedy)	6.70	8	-0.04	11
ϵ = 0.1	7.71	8	6.82	11
ϵ = 0.2	7.79	8	8.23	11
ϵ = 0.5	7.73	8	9.40	11

Table 4: Average Q-values for α = 1/k

Epsilon-greedy	Average of action value Q(a1) of 100 runs	True action value Q∗(a1)	Average of action value Q(a2) of 100 runs	True action value Q∗(a2)
ϵ = 0 (greedy)	2.16	8	6.86	11
ϵ = 0.1	4.02	8	7.45	11
ϵ = 0.2	5.40	8	7.87	11
ϵ = 0.5	6.94	8	9.81	11

Comments:
Overall, the graph for α = 1/k seemed to perform better on average, likely due to equal weight being distributed to all observations. As for the epsilon values, ϵ = 0 seems to outperform the

rest. It seems like avoiding exploration seems to be key in this case as the plot curve seems to quickly stabilize at around 200 iterations.

Part 2

Main coding part:

```
class EpsilonGreedy(UtilityFunctions):

    """Part 2"""
    def part2(self):
        all_results = []
        for (i, j) in self.part2_Q_a1_a2_values:
            average_Q_a1_values_0, average_Q_a2_values_0, average_acc_rewards = self.rl_bandit_100loops(self.part2_combina
            all_results.append({'label': f'Q_a1 = {i}, Q_a2 = {j}', 'data': average_acc_rewards[0]})
        self.part2_results = all_results

        plt.figure(figsize=(10, 6))

        colors = ['blue', 'orange', 'green']

        for result, color in zip(all_results, colors):
            plt.plot(range(1, 1001), result['data'],
                     label=result['label'], color=color, linewidth=2)

        plt.xlabel('Time(t)', fontsize=12)
        plt.ylabel('Average Accumulated Reward', fontsize=12)
        plt.title('Epsilon-greedy with different Initial Q-values (α=0.1, ε=0.1)', fontsize=14, fontweight='bold')
        plt.legend(loc='best')
        plt.grid(True, alpha=0.3)
        plt.tight_layout()
        plt.savefig('plots/part2_results.png', dpi=300, bbox_inches='tight')
        plt.close()
```

Figure 2: Average accumulated reward in respect to step/time:

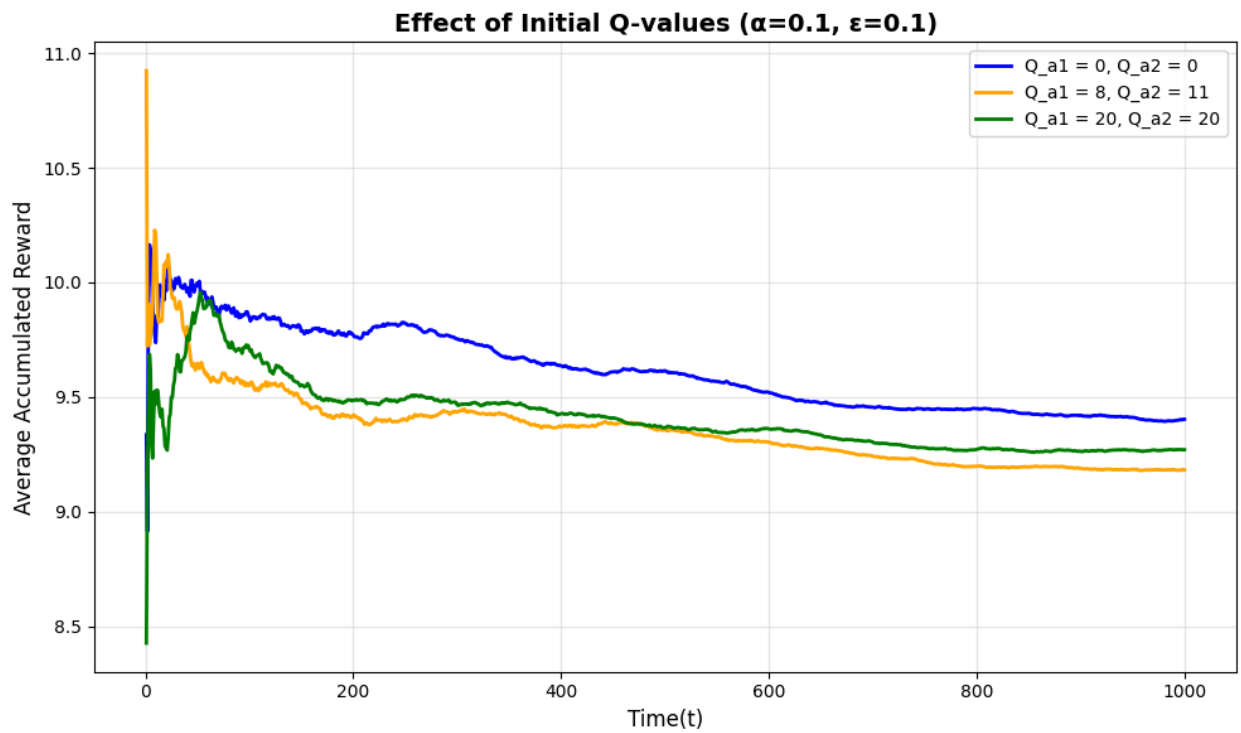


Table 5: Average Q-values for different optimistic initializations

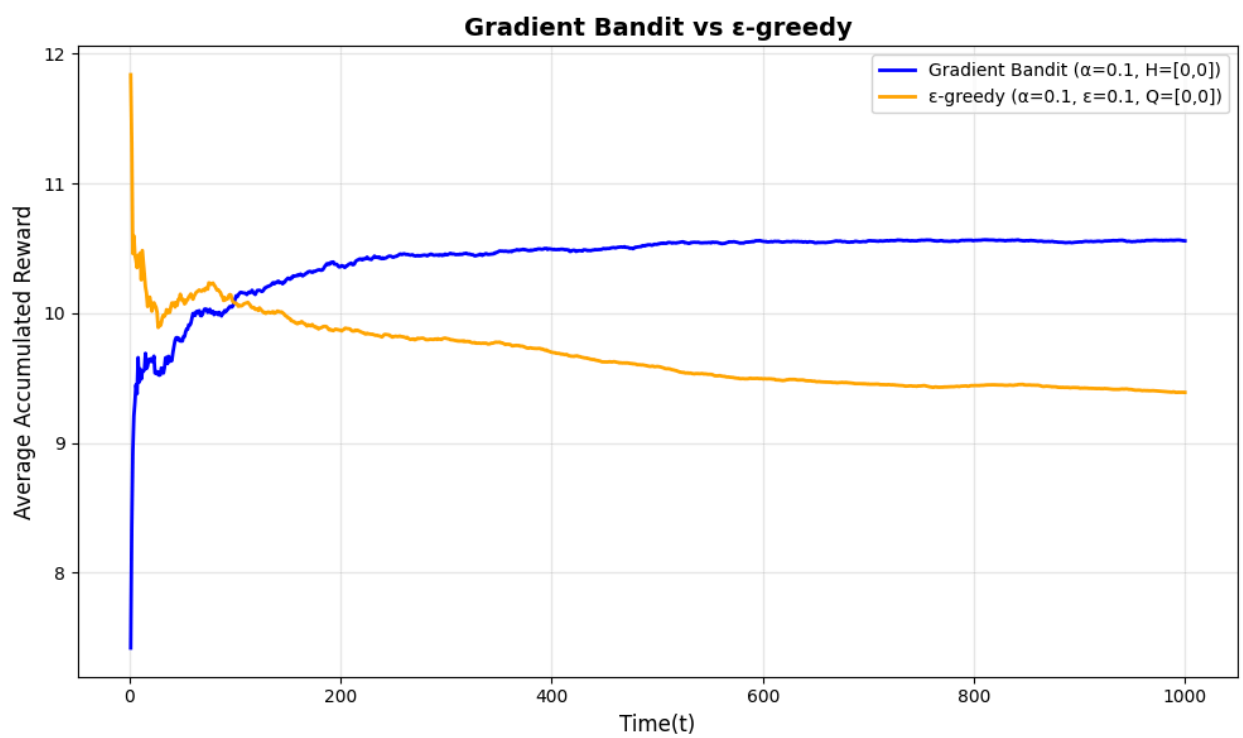
Epsilon-greedy	Average of action value Q(a1) of 100 runs	True action value Q∗(a1)	Average of action value Q(a2) of 100 runs	True action value Q∗(a2)
Q = [0 0]	7.41	8	8.53	11
Q = [8 11]	7.53	8	7.74	11
Q = [20 20]	7.51	8	8.12	11

Comments:
The graph above shows that, if all other parameters are constant, the higher the initial values are, the more the accumulated reward fluctuates at the start as higher initial values lead to a more optimistic start and higher exploration initially, compared to lower ones, before converging due to exploitation.

Part 3
Main coding part:

```python
"""Part 3"""
def part3(self, part2):
    average_H_a1_values, average_H_a2_values, average_acc_rewards_gradient = self.rl_gradient_bandit_100loops(self.

    plt.figure(figsize=(10, 6))

    # Plot Gradient Bandit
    plt.plot(range(1, 1001), average_acc_rewards_gradient,
             label='Gradient Bandit (α=0.1, H=[0,0])',
             color='blue', linewidth=2)

    # Plot ε-greedy
    plt.plot(range(1, 1001), part2[0]['data'],
             label='ε-greedy (α=0.1, ε=0.1, Q=[0,0])',
             color='orange', linewidth=2)

    plt.xlabel('Time(t)', fontsize=12)
    plt.ylabel('Average Accumulated Reward', fontsize=12)
    plt.title('Gradient Bandit vs ε-greedy', fontsize=14, fontweight='bold')
    plt.legend(loc='best')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('plots/part3_comparison.png', dpi=300, bbox_inches='tight')
    plt.close()
```

Figure 3: Comparison of gradient-bandit policy and epsilon-greedy policy



Comments:

Gradient Bandit seems to outperform ε-greedy, likely due to adaptive exploration. While ε-greedy maintains fixed 10% random exploration throughout, Gradient Bandit's softmax policy focuses on probability on better actions as learning progresses. After identifying the best action, Gradient Bandit exploits it majority of the time, whereas ε-greedy still wastes 10% of the time on exploration.

Part 4

Main coding part:

```
def part4(self, epsilon_greedy_results, average_acc_rewards_gradient):
    c_values_UCB_result = {}
    for c in self.c_values:
        average_Q_a1_values_UCB, average_Q_a2_values_UCB, average_acc_rewards_UCB = self.rl_bandit_100loops_UCB(c)
        c_values_UCB_result[c] = average_acc_rewards_UCB

    self.part4_results = c_values_UCB_result

    plt.figure(figsize=(10, 6))

    colors = ['blue', 'orange', 'green']
    for c, color in zip(self.c_values, colors):
        plt.plot(range(1, 1001), c_values_UCB_result[c],
                 label=f'UCB c={c}', color=color, linewidth=2)

    plt.xlabel('Time(t)', fontsize=12)
    plt.ylabel('Average Accumulated Reward', fontsize=12)
    plt.title('UCB Performance for Different c Values', fontsize=14, fontweight='bold')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('plots/part3_UCB_results.png', dpi=300, bbox_inches='tight')
    plt.close()

    comparable_results = {
        'UCB c=5': c_values_UCB_result[5],
        'ε-greedy α = 0.9^k ε=0.5': epsilon_greedy_results[('0.9**k', 0.5)],
        'gradient-bandit': average_acc_rewards_gradient
    }
```

Figure 4: Graph showing average accumulated reward across time for UCB with three c values

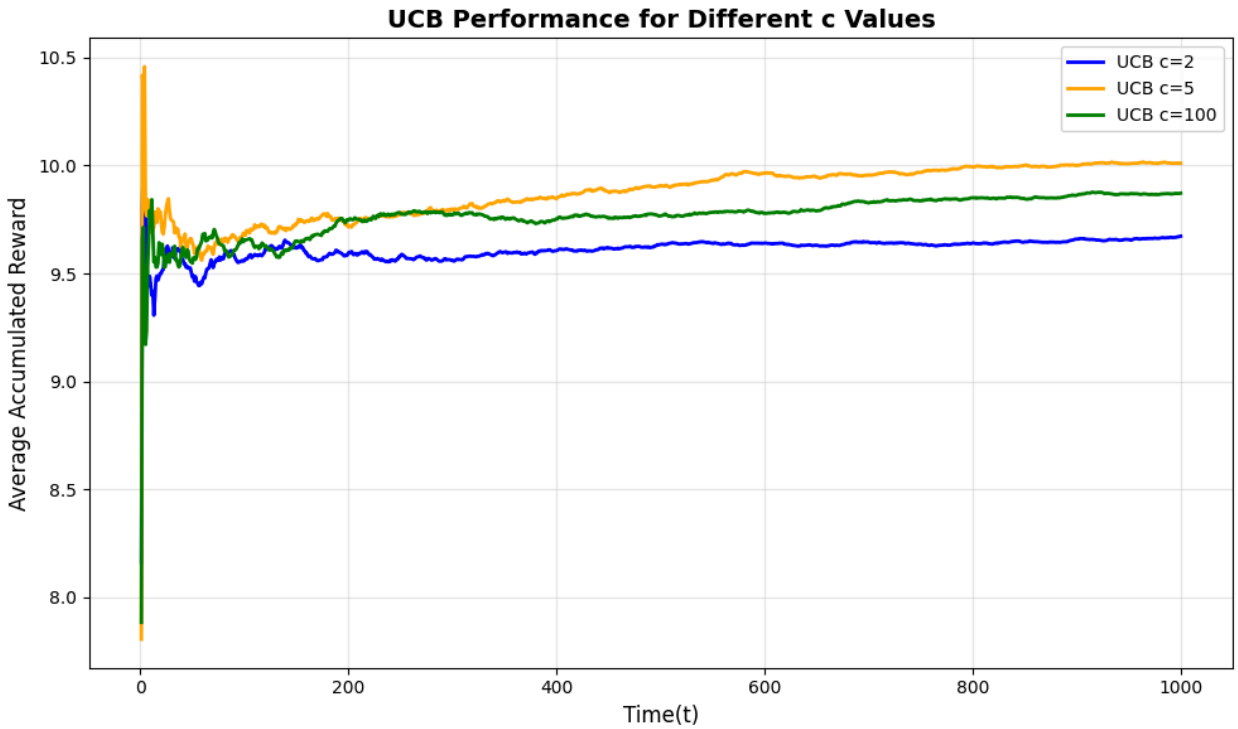
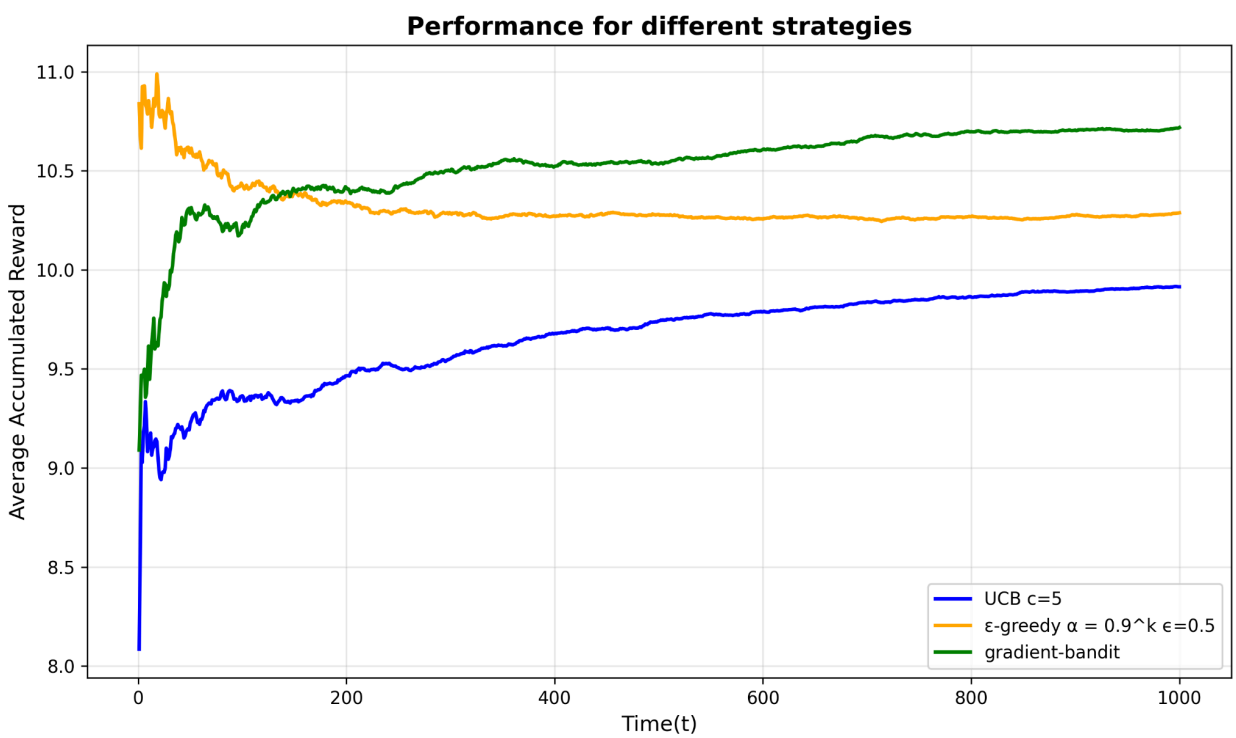


Figure 5: Graph comparing best-performing UCB (c=5), best-performing ϵ-greedy policy, and the Gradient-Bandit policy



Comments:

Out of the three strategies employed, gradient bandit performs the best, followed by the best-performing epsilon-greedy, and then the best-performing UCB. Gradient bandit performs best due to its least wastage on exploration, while epsilon-greedy shows strong early performance but declines steadily due to its excessive 50% exploration. UCB performs the worst but it shows steady learning so, it might be due to c = 5 being too conservative. However, we learned in the previous plot that c=100 performed worse than c = 5 so, an optimal value for c is likely to be between 5 and 100.